

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

Online Quiz System

Console-Based Multiple-Choice Quiz Game

Project	Online Quiz System
Version	1.0
Programming Language	C / C++
Authors	Team Member 1 - Madhusudhan Team Member 2 - Sarah Team Member 3 - Thanvitha Team Member 4 - Jishnu
Date	03-10-2026
Status	Draft / For Submission

Project Description: A console-based quiz game with multiple-choice questions, input validation, automatic scoring, result display, replay, and clean exit.

Revision History

Version	Date	Author	Change Summary
0.1	03-10-2026	Team Member 1 - Madhusudhan Team Member 2 - Sarah Team Member 3 - Thanvitha Team Member 4 - Jishnu	Initial architecture and design draft
1.0	03-10-2026	Team Member 1 - Madhusudhan Team Member 2 - Sarah Team Member 3 - Thanvitha Team Member 4 - Jishnu	Completed SAD with component and sequence diagrams

Approvals

Role	Name	Signature / Date
Student / Author 1	Madhusudhan / PES2UG24CS210	
Student / Author 2	Jishnu / PES2UG24CS204	
Student / Author 3	Sarah / PES2UG24CS224	
Student / Author 4	Thanvitha/PES2UG24CS197	
Faculty / Evaluator		

1. Introduction

1.1 Purpose

This document specifies the architecture and detailed design of the Online Quiz System. It translates the requirements in the Software Requirements Specification into implementable software components, interfaces, data structures, execution flows, security controls, and error-handling decisions for a console-based C/C++ application.

1.2 Scope

The design covers quiz initialization, player-name capture, local question-bank loading and validation, sequential MCQ presentation, answer validation, answer evaluation, score calculation, progress display, final percentage and result summary, replay, and clean termination. The system runs locally in a terminal and does not require a GUI, web server, database server, cloud service, or network connection.

1.3 Audience

- Students and developers implementing the project in C or C++.
- Faculty members and evaluators reviewing software-engineering artefacts.
- Testers validating the system against the SRS and Software Test Plan.
- Maintainers updating source code or the local question bank.

1.4 Definitions

Term	Meaning
MCQ	Multiple-Choice Question with one stored correct answer.
CLI	Command-Line Interface used for all player interaction.
SRS	Software Requirements Specification.
STP	Software Test Plan.
RTM	Requirements Traceability Matrix.
Question Bank	Local or embedded collection of question text, options, and correct-answer references.
Session / Attempt	One complete quiz run from initialization to result summary.
Component	A logical module with a clear responsibility and interface.

2. Document Overview

2.1 How to Use This Document

Section 3 defines the system architecture, architectural goals, stakeholders, component structure, technology choices, risks, traceability, and security architecture. Section 4 defines the implementation-level design, two required sequence diagrams, internal component interfaces, error handling, console UX, and future enhancements. Section 5 provides supporting glossary, references, and tools.

2.2 Related Documents

- Online Quiz System - Software Requirements Specification (SRS) v1.0.
- Online Quiz System - Software Test Plan (STP) v1.0.
- Requirements Traceability Matrix (RTM) included in the SRS.
- Project source code and test-case specifications.

3. Architecture

3.1 Goals & Constraints

The architecture is intentionally simple and modular because the project is a single-process console application. The main goals are correctness, predictable scoring, robustness against invalid input, portability, maintainability, and clear separation of responsibilities.

Goal	Architectural Response
Correctness	Each accepted answer must be compared with the correct question key and reflected accurately in the score and percentage.
Robustness	Invalid, empty, malformed, or out-of-range input must not crash the application or corrupt session state.
Maintainability	Question loading, input validation, quiz flow, scoring, and result display are separated into clear modules/functions.
Portability	The implementation should rely on standard C11 or C++17 facilities wherever possible.
Performance	For the intended local question-bank size, startup and interaction should remain within the SRS response-time targets.
Security / Integrity	User input is treated only as data; array/container bounds, question records, and score limits are checked before use.

Key constraints:

- The primary interface is text based and runs in a terminal/command prompt.
- Implementation language is C or C++.
- One player is supported per running process.
- Each question has a finite list of options and exactly one stored correct-answer reference.
- Question data is embedded in source code or loaded from a local file; no database server is required.
- No network or inter-process communication is required by the base version.

3.2 Stakeholders & Concerns

Stakeholder	Primary Concerns
Player	Clear prompts, correct scoring, easy recovery from invalid input, readable result summary.
Developer / Maintainer	Modular code, simple interfaces, easy question-bank updates, portable build.
Tester	Deterministic behavior, observable outputs, clear failure messages, requirement traceability.
Faculty / Evaluator	Conformance to SRS/STP, completeness of design artefacts, demonstrable architecture.

3.3 Component (UML) Diagram

The system is organized as a modular monolithic application. The Console UI is the single entry point for the player, while the remaining components isolate validation, quiz flow, data access, scoring, session state, and result formatting.

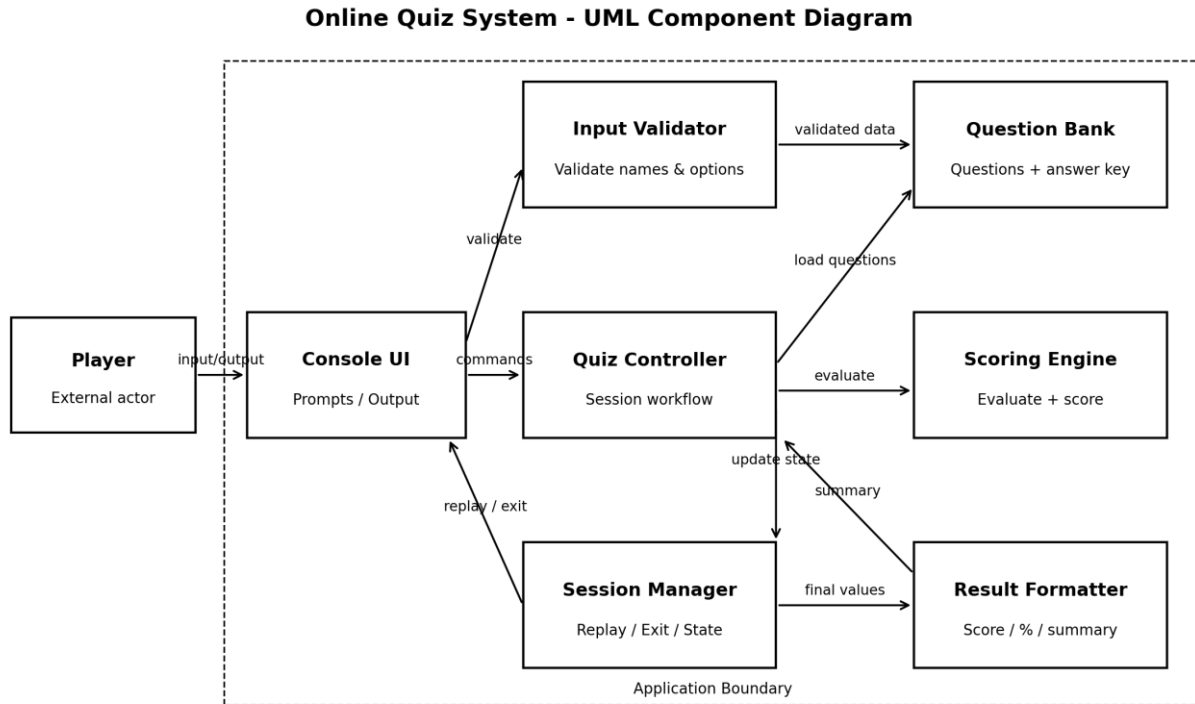


Figure 1. UML component diagram for the Online Quiz System.

3.4 Component Descriptions

Component	Responsibility
Console UI	Displays welcome screen, questions, options, feedback, result summary, replay/exit choices, and error messages. Captures raw keyboard input.
Input Validator	Checks player name and answer/control input before the values are used. Rejects empty, malformed, and out-of-range inputs.
Quiz Controller	Coordinates the quiz lifecycle: initializes a session, obtains questions, presents one question at a time, records accepted answers, and decides when the quiz ends.
Question Bank	Stores or loads question text, option lists, and correct-answer references. Validates that records are complete and internally consistent.
Scoring Engine	Compares the accepted option with the stored correct answer, updates the score, and calculates the final percentage.
Session Manager	Holds attempt state such as player name, current question index, accepted responses, and score. Resets state for replay and closes state on exit.
Result Formatter	Builds the final display containing player name, score, total questions, percentage, and completion message.

3.5 Chosen Architecture Pattern and Rationale

A layered modular-monolith architecture is selected. Presentation logic is kept in the Console UI, validation is isolated, application flow is controlled by the Quiz Controller and Session Manager, and domain logic such as question handling and scoring is separated into dedicated components. This provides separation of concerns without introducing deployment complexity.

Alternative considered: a client-server or microservices architecture was rejected because the base project is a local, single-user console application with no network requirement. Splitting the system into independently deployed services would add configuration, communication, and failure modes without improving the required functionality.

Architecture decision: use direct in-process function or method calls between components. This keeps execution deterministic, easy to test, and suitable for both C and C++ implementations.

3.6 Technology Stack & Data Stores

Item	Choice
Programming language	C11 or later, or C++17 or later.
Runtime / libraries	Standard C/C++ runtime and standard library. No mandatory third-party framework.
User interface	Console / terminal using standard input and output.
Question storage	Either compile-time arrays/structures or a local text/CSV-style file, depending on implementation choice.
Session storage	In-memory variables/structures for player name, current index, accepted responses, score, and total questions.
Persistent database	Not required for the base version.
Networking	Not used.

3.7 Risks & Mitigations

Risk	Mitigation
Malformed console input causes incorrect state or crash	Validate every input before conversion or use; re-prompt without advancing the question.
Malformed question record	Validate question text, options, and correct-answer reference before the question can be used.
Out-of-bounds array/container access	Check indexes against question and option counts before dereferencing.
Incorrect scoring logic	Centralize answer evaluation and score updates in the Scoring Engine and cover it with unit tests.
Replay retains previous score or index	Session Manager performs an explicit state reset before a new attempt.
Platform/compiler differences	Use standard C11/C++17 facilities and isolate any platform-specific code if it becomes necessary.
Question bank unavailable	Display a clear load error and prevent quiz execution rather than using incomplete data.

3.8 Traceability to Requirements

SRS Requirement(s)	Responsible Component(s)
OQS-F-001, OQS-F-002, OQS-F-006 to OQS-F-009, OQS-F-013, OQS-F-014	Console UI + Input Validator
OQS-F-003, OQS-F-010, OQS-F-015, OQS-F-019, OQS-F-020	Session Manager + Quiz Controller
OQS-F-004, OQS-F-005	Question Bank
OQS-F-011, OQS-F-012, OQS-F-016, OQS-F-017	Scoring Engine
OQS-F-018	Result Formatter
OQS-NF-001, OQS-NF-002	Quiz Controller + Console UI + Question Bank
OQS-NF-003, OQS-NF-004	Input Validator + Console UI + Session Manager
OQS-NF-005	Overall modular architecture and standard C/C++ technology choice
OQS-SR-001	Input Validator
OQS-SR-002	Quiz Controller + Question Bank
OQS-SR-003	Question Bank
OQS-SR-004	Console UI + Input Validator
OQS-SR-005	Scoring Engine + Session Manager

3.9 Security Architecture

The system has no remote attack surface in the base version, so security focuses on safe handling of local input, question integrity, memory/index safety, command safety, and score integrity.

Threat / Failure Mode	Control	Requirement
Invalid or hostile console input	Input Validator checks type, length/format, and allowed option range before the value reaches quiz logic.	OQS-SR-001
Out-of-range index or container access	Quiz Controller and Question Bank verify indexes before accessing questions/options.	OQS-SR-002
Invalid answer reference in question data	Question Bank rejects a record whose stored correct answer is not one of its displayed options.	OQS-SR-003
Command/code injection through user text	Console UI treats input only as data; no shell/system execution or dynamic evaluation is performed.	OQS-SR-004

Threat / Failure Mode	Control	Requirement
Score manipulation or stale replay state	Scoring Engine enforces $0 \leq \text{score} \leq \text{total questions}$; Session Manager resets all scoring state on replay.	OQS-SR-005

4. Design

4.1 Design Overview

The design follows a simple request-response loop in which user input is validated before it changes the session. At startup, the application initializes state and loads/validates the question bank. For each question, the UI prints the question and options, validates one answer, sends the accepted choice for evaluation, updates score/progress, and advances. After the final question, the application calculates percentage and shows the summary. The player then chooses replay or exit.

Step	Design Behavior
1. Start	Initialize application and display welcome/start prompt.
2. Identify Player	Read and validate a non-empty player name.
3. Prepare Quiz	Reset session state and load/validate the question bank.
4. Present Question	Display one active question, option labels, and progress.
5. Validate Answer	Accept only a documented option value; otherwise re-prompt.
6. Evaluate	Compare with answer key and update score if correct.
7. Repeat	Advance until all questions are completed.
8. Finish	Calculate percentage and display final result summary.
9. Control	Replay with a fresh state or terminate cleanly.

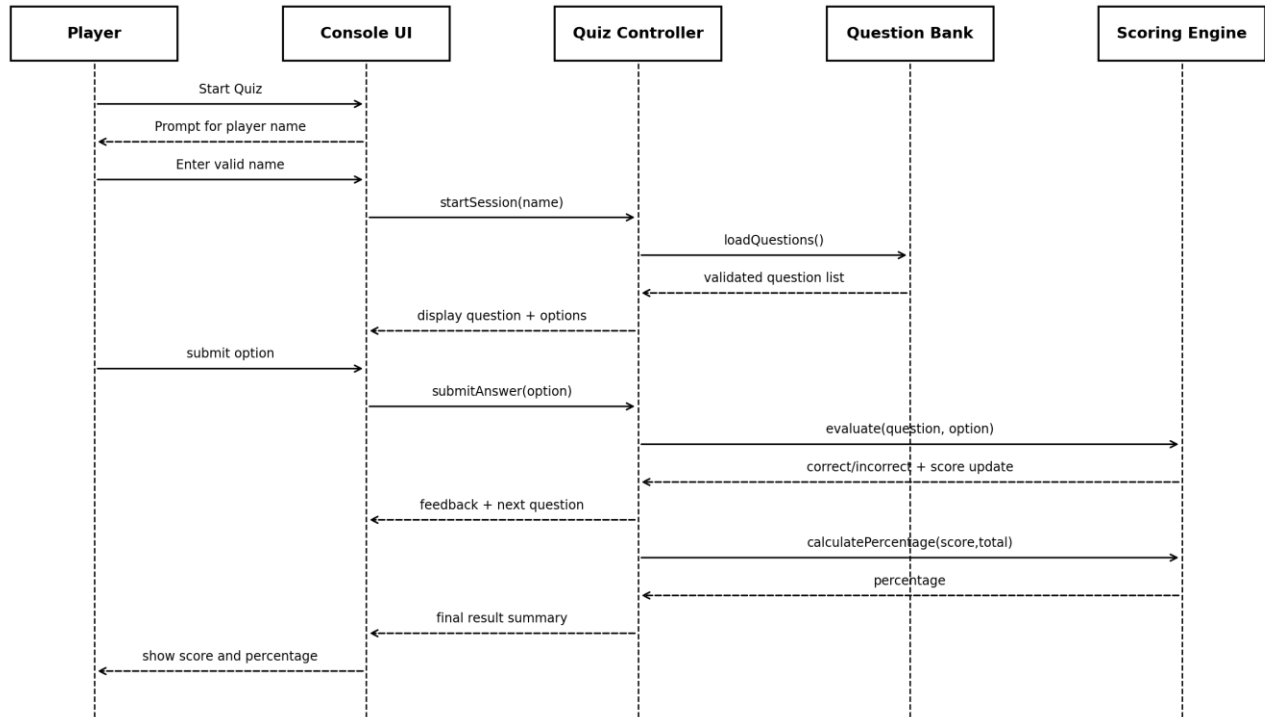
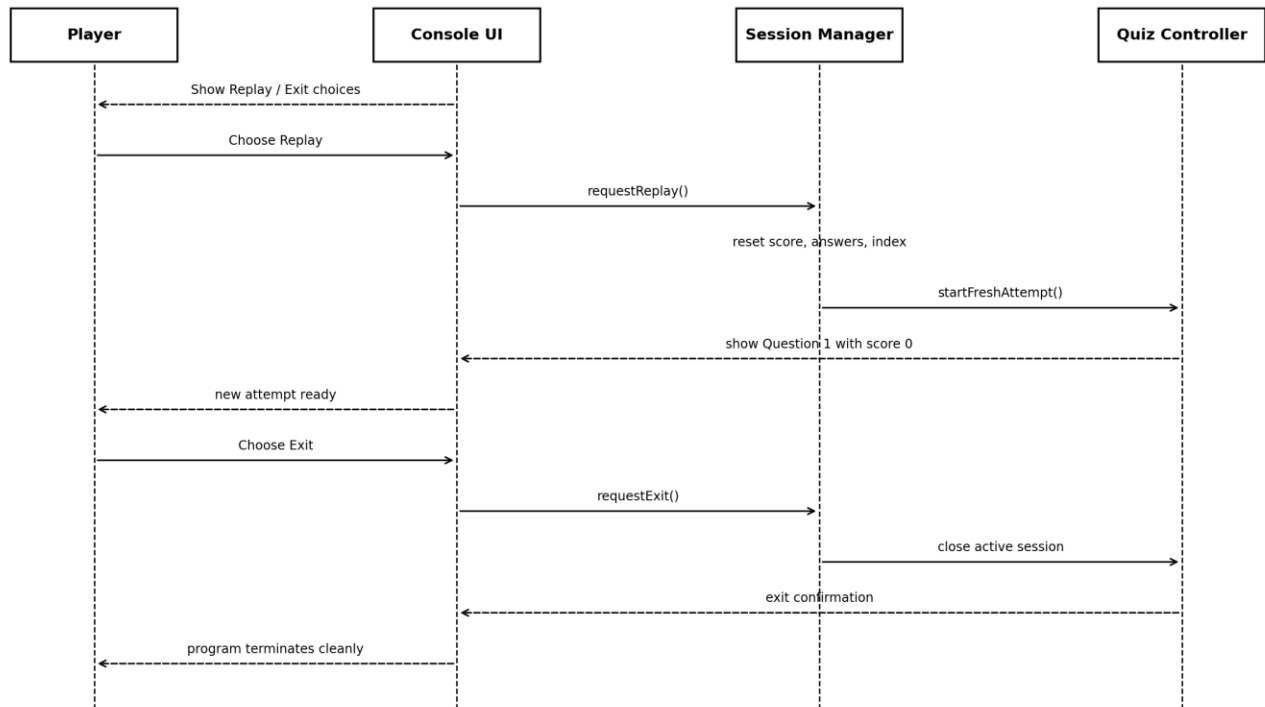
4.1.1 Core Data Structures

The exact syntax depends on whether C or C++ is selected, but the following logical records are required.

Logical Structure	Key Fields
Question	id or index; questionText; options[]; correctAnswer; optional category/metadata.
SessionState	playerName; currentQuestionIndex; score; totalQuestions; acceptedResponses[]; completed flag.
ResultSummary	playerName; score; totalQuestions; percentage.

4.2 UML Sequence Diagrams

The template requires at least two sequence diagrams. The first covers the primary quiz flow; the second covers replay and clean exit.

Sequence Diagram 1 - Take Quiz and Calculate Result*Figure 2. Sequence diagram - take quiz, evaluate answers, and calculate the result.***Sequence Diagram 2 - Replay and Exit Control***Figure 3. Sequence diagram - replay the quiz or exit the application.*

4.3 API Design

The Online Quiz System has no network API. This section therefore defines internal component interfaces (functions/methods) between modules, which satisfies the template requirement for interface definitions while remaining consistent with the console-only SRS.

4.3.1 Question Bank Interface

Operation	Purpose	Return	Errors / Validation
loadQuestions()	Loads or initializes the configured questions.	Question collection + status	Missing/unreadable source; malformed record.
validateQuestion(question)	Checks question text, option count/content, and correct-answer reference.	true / false	Invalid or incomplete record.
getQuestion(index)	Returns one validated question at a valid index.	Question	Index outside [0, questionCount-1].

4.3.2 Scoring Engine Interface

Operation	Purpose	Return	Errors / Validation
evaluateAnswer(question, selectedOption)	Compares accepted choice with stored answer key.	true if correct; false otherwise	selectedOption must already be validated.
updateScore(currentScore, isCorrect)	Adds one point only when the answer is correct.	new score	Result must remain within valid bounds.
calculatePercentage(score, totalQuestions)	Calculates (score / totalQuestions) x 100.	percentage	totalQuestions must be greater than zero.

4.3.3 Session Control Interface

Operation	Purpose	Return	Errors / Validation
startSession(playerName)	Creates a fresh attempt with score 0 and question index 0.	Session state	Reject empty/invalid name before call.
recordResponse(index, selectedOption)	Stores the single accepted answer for the current question.	success / failure	Reject duplicate or invalid index.
resetSession()	Clears prior answers, score, and progress before replay.	reset state	No previous attempt data may remain.
endSession()	Closes the active attempt and allows clean termination.	status	No unfinished prompt should remain.

4.4 Error Handling, Logging & Monitoring

Condition	Handling
Empty player name	Display a short message and re-prompt without starting the quiz.
Invalid answer option	Display accepted option format and re-prompt; do not change score or index.
Question bank cannot be loaded	Display a clear load error and stop quiz execution safely.
Malformed question record	Reject the invalid record/question before it is presented.
Out-of-range internal index	Prevent access, report an internal error if necessary, and avoid undefined behavior.
Division-by-zero risk	Do not calculate percentage when totalQuestions is zero.
Unexpected end of input	Terminate or return to a safe state without crashing.

Logging and monitoring are lightweight because the application is a local console program. User-facing errors are printed to the console. During development and testing, diagnostic output may be written to standard error or captured in test logs. The base design does not require background monitoring, remote telemetry, or collection of sensitive user data.

4.5 UX Design

The user experience is intentionally minimal and consistent. Each screen states what the player should do next and what input format is accepted.

Screen / State	UX Requirement
Launch	Display project name and Start Quiz option.
Player Name	Prompt for name; reject empty input clearly.
Question Screen	Show question number, question text, all available option labels, and expected answer format.
Invalid Input	Show one concise error and repeat the same prompt without consuming the question.
Answer Feedback	Display Correct or Incorrect after each accepted answer.
Progress	Show current question number and total question count.
Result Summary	Display player name, final score, total questions, and percentage.

Screen / State	UX Requirement
Session Control	Offer Replay and Exit choices using the same consistent input format.

4.6 Open Issues & Next Steps

The following are future enhancements and are not required for the current base version unless the implementation is intentionally extended:

- Randomize question order and option order.
- Add categories, difficulty levels, or configurable question count.
- Persist high scores or previous attempts in a local file.
- Add a per-question timer.
- Add an administrator mode for maintaining the question bank.
- Provide a GUI or web front end while retaining the same scoring and validation modules.
- Add a database only if persistent multi-user functionality is introduced.

5. Appendices

5.1 Glossary

- MCQ - Multiple-Choice Question.
- CLI - Command-Line Interface.
- SRS - Software Requirements Specification.
- STP - Software Test Plan.
- RTM - Requirements Traceability Matrix.
- UML - Unified Modeling Language.
- Module - A source-level unit with a defined responsibility and interface.
- Session - One complete quiz attempt.

5.2 References

- Online Quiz System SRS v1.0.
- Online Quiz System STP v1.0.
- C11 / C++17 language and standard-library documentation applicable to the chosen implementation.
- Course software-engineering templates and evaluation guidelines.

5.3 Tools

- C/C++ compiler and IDE/editor used by the project team.
- PlantUML, draw.io, or equivalent UML diagramming tool (design documentation).
- Git/GitHub for source version control, if used by the team.
- Command prompt / terminal for build, execution, and testing.